Unidad Productiva Pedagógica Jesús Obrero

IUJO

Extensión Barquisime

NIVEL MASTER - SEMANA 4

ARQUITECTURA Y ROBUSTEZ

Hacia Sistemas Empresariales en Java

Aprenderemos a diseñar contratos de software, manejar errores de forma elegante y crear contenedores de datos inteligentes.

1. CLASES ABSTRACTAS: EL PLANO MAESTRO

```java
public abstract class Usuario {
    protected String nombre;

    public Usuario(String n) { this.nombre = n; }

    // Método común con código
    public void saludar() {
        System.out.println("Accediendo como: " + nombre);
    }

    // Método abstracto (OBLIGATORIO)
    public abstract void permisos();
}
```

Unidad Productiva Pedagógica Jesús Obrero

IUJO Extensión Barquisime

HERENCIA

IMPLEMENTANDO LA ABSTRACCIÓN

```java
public class Estudiante extends Usuario {
    public Estudiante(String n) { super(n); }

    @Override
    public void permisos() {
        System.out.println("Nivel: Solo Lectura.");
    }
}
```

2. INTERFACES: CONTRATOS DE HABILIDADES

```java
public interface Autenticable {
    boolean login(String pass);
}

public class Admin implements Autenticable {
    @Override
    public boolean login(String p) {
        return p.equals("admin123");
    }
}
```

EJECUCIÓN DE CONTRATOS

```java
public static void main(String[] args) {
    Autenticable auth = new Admin();
    boolean exito = auth.login("admin123");
    System.out.println("¿Acceso concedido? " + exito);
}
```

```bash
$ java PruebaInterfaces
> ¿Acceso concedido? true
```

Fíjate que usamos el tipo de la interfaz **Autenticable** para crear la variable. ¡Esto es polimorfismo!

Unidad Productiva Pedagógica Jesús Obrero

IUJO Extensión Barquisimeto

POLIMORFISMO

3. EXCEPCIONES: EL MURO DE SEGURIDAD

```java
public class Sistema {
    public void validarCupo(int inscritos) throws Exception {
        if (inscritos >= 30) {
            throw new Exception("Sección llena");
        }
        System.out.println("Inscripción exitosa");
    }
}
```

ATRAPANDO EXCEPCIONES EN EL MAIN

```java
public static void main(String[] args) {
    Sistema s = new Sistema();
    try {
        s.validarCupo(35); // Esto lanzará el error
    } catch (Exception e) {
        System.err.println("ALERTA: " + e.getMessage());
    }
}
```

```bash
$ java PruebaSistema
> ALERTA: Sección llena
```

4. EXPRESIONES REGULARES (REGEX)

```java
public class ValidadorIUJO {
    public static boolean esEmailValido(String mail) {
        // Estructura: texto + @iujo.edu.ve
        return mail.matches("^[\\w.-]+@iujo\\.edu\\.ve$");
    }
}
```

PROBANDO EL FILTRO IUJO

```java
public static void main(String[] args) {
    String m1 = "estudiante@iujo.edu.ve";
    String m2 = "hacker@gmail.com";

    System.out.println(m1 + " ¿Es válido? " + ValidadorIUJO.esEmailValido(m1));
    System.out.println(m2 + " ¿Es válido? " + ValidadorIUJO.esEmailValido(m2));
}
```

```bash
$ java PruebaRegex
> estudiante@iujo.edu.ve ¿Es válido? true
> hacker@gmail.com ¿Es válido? false
```

5. GENÉRICOS: MOLDES UNIVERSALES

```
import java.util.*;

public class Caja<T> {
    private T contenido;

    public void meter(T c) { this.contenido = c; }
    public T sacar() { return contenido; }
}
```

EJECUCIÓN CON DIFERENTES TIPOS

```java
public static void main(String[] args) {
    Caja<String> cajaTexto = new Caja<>();
    cajaTexto.meter("Certificado IUJO");

    Caja<Integer> cajaNota = new Caja<>();
    cajaNota.meter(20);

    System.out.println("Sacando: " + cajaTexto.sacar());
    System.out.println("Nota final: " + cajaNota.sacar());
}
```

```bash
$ java PruebaGenericos
> Sacando: Certificado IUJO
> Nota final: 20
```

RETO: POLIMORFISMO EN EL ZOOLÓGICO

SOLUCIÓN: RETO DEL ZOOLÓGICO

```java
import java.util.ArrayList;
import java.util.List;

abstract class Animal {
    public abstract String hacerSonido();
}

class Perro extends Animal {
    @Override
    public String hacerSonido() { return "Guau"; }
}

class Gato extends Animal {
    @Override
    public String hacerSonido() { return "Miau"; }
}

class Ave extends Animal {
    @Override
    public String hacerSonido() { return "Pío"; }
}

public class Zoologico {
    public static void main(String[] args) {
        List<Animal> animales = new ArrayList<>();
        animales.add(new Perro());
        animales.add(new Gato());
        animales.add(new Ave());

        for (Animal animal : animales) {
            System.out.println("El animal hace: " + animal.hacerSonido());
        }
    }
}
```

¡MISIÓN CUMPLIDA!

Has dominado los 5 pilares de la arquitectura profesional.

PREPÁRATE PARA EL PROYECTO FINAL EN LA SEMANA 5